

Metodología de 6 Pasos para el Diseño y Codificación de Agentes de IA

ELECTRONICA

27 de junio de 2026

Índice

1. Introducción	2
2. Mapa de Adaptación	2
3. Desglose de Cada Paso	2
3.1. PRD — <i>Product Requirements Document</i>	2
3.2. TRD — <i>Technical Requirements Document</i>	3
3.3. Diseño UI/UX	3
4. Arquitectura de Núcleo: El Bucle Orquestador-LLM	3
4.1. Responsabilidades del Orquestador	4
4.2. App Flow	4
4.3. Esquema Backend	5
4.4. Plan de Implementación	5
5. Conclusión	5

1. Introducción

La construcción de agentes de inteligencia artificial requiere un enfoque estructurado que garantice claridad, mantenibilidad y resultados predecibles. La siguiente metodología adapta los pasos clásicos del desarrollo de software (PRD, TRD, UI/UX, App Flow, Esquema Backend, Plan de Implementación) al contexto específico de agentes de IA.

2. Mapa de Adaptación

Paso	En App Convencional	En Agente de IA
PRD	Funcionalidades de usuario	Capacidades del agente, tools que necesita, límites
TRD	Stack web (React, Django, etc.)	Modelo/API, vector store, framework de agente (LangChain, CrewAI), tools, memoria
Diseño UI/UX	Pantallas y componentes	Interfaz de interacción (CLI, chat, web) y <i>experience</i> del agente (system prompt, tone)
App Flow	Navegación entre vistas	Flujo de razonamiento → tool call → validación → respuesta (con <i>humans-in-the-loop</i> si aplica)
Esquema Backend	DB, endpoints, lógica	System prompt, tools/skills, memoria (RAG/Chroma), orquestación multi-agente
Plan Impl.	Sprints por feature	Iterar: prompt → tool → validar → registrar errores en memoria → refinar

Cuadro 1: Adaptación de la metodología de 6 pasos al desarrollo de agentes de IA.

3. Desglose de Cada Paso

3.1. PRD — *Product Requirements Document*

Define el **qué** y el **para quién** del agente.

- **Propósito:** ¿Qué problema resuelve el agente?
- **Capacidades:** Lista de habilidades concretas (responder preguntas técnicas, generar código, evaluar exámenes, etc.).
- **Tools necesarias:** Acceso a web, sistema de archivos, APIs externas, bases de datos vectoriales.

- **Límites:** Restricciones explícitas (no ejecutar código arbitrario, no acceder a ciertos directorios, etc.).
- **Usuarios:** ¿Quién interactúa con el agente? ¿Desarrolladores, estudiantes, operadores?

3.2. TRD — *Technical Requirements Document*

Define el **cómo** desde la perspectiva técnica.

- **Modelo base:** LLM seleccionado (GPT-4, Llama 3, Groq, etc.) y parámetros de inferencia.
- **Framework de agente:** LangChain, CrewAI, AutoGen, o implementación propia.
- **Vector store:** ChromaDB, Pinecone, Weaviate para memoria a largo plazo (RAG).
- **Memoria conversacional:** Buffer de historial, resumen, o memoria híbrida.
- **Stack de ejecución:** Entorno Conda, contenedores Docker, sandboxing.
- **APIs externas:** Conexiones a servicios (Groq, OpenRouter, Google Gemini, etc.).

3.3. Diseño UI/UX

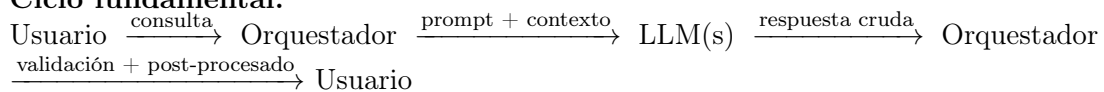
Define la **interfaz de interacción** y la **personalidad** del agente.

- **Interfaz:** CLI, chat web (Streamlit/Gradio), integración en editor (VS Code), o API.
- **System prompt:** Tono, rol, reglas de comportamiento y restricciones.
- **Feedback:** Indicadores de progreso, manejo de errores amigable, confirmaciones antes de acciones destructivas.
- **Accesibilidad:** Multi-idioma, formato de respuestas (markdown, LaTeX, texto plano).

4. Arquitectura de Núcleo: El Bucle Orquestador-LLM

El corazón de un agente de IA es un **script orquestador** que actúa como middleware entre el usuario y uno o varios modelos de lenguaje (LLMs). Este orquestador no es un mero enrutador: construye dinámicamente los prompts con contexto relevante (memoria, herramientas disponibles, historial), decide qué LLM invocar según la tarea, ejecuta las llamadas a las APIs, valida y post-procesa las respuestas, y finalmente presenta el resultado al usuario.

Ciclo fundamental:



4.1. Responsabilidades del Orquestador

1. Recibir la consulta del usuario.
2. Enriquecerla con contexto: memoria a largo plazo (RAG), historial conversacional, herramientas disponibles, directivas del proyecto.
3. Decidir qué LLM o secuencia de LLMs invocar (ej. Groq para rapidez, Gemini para multimodal, OpenRouter como respaldo).
4. Construir el prompt final con system instructions + contexto + consulta.
5. Ejecutar la(s) llamada(s) a la(s) API(s) del LLM.
6. Validar la respuesta cruda: esquema JSON, coherencia, integridad.
7. Post-procesar: limpiar trailing commas, extraer bloques, formatear salida.
8. Actualizar el estado y la memoria con el resultado.
9. Devolver la respuesta al usuario.

4.2. App Flow

Define el **flujo de razonamiento y acción** completo del agente.

■ Ciclo principal (detallado):

1. Recibir entrada del usuario.
2. Consultar memoria (RAG) para contexto relevante.
3. Construir prompt enriquecido: system prompt + herramientas + memoria + consulta.
4. Seleccionar LLM según tipo de tarea (texto, imagen, código).
5. Ejecutar llamada a la API del LLM.
6. Validar y sanitizar la respuesta del LLM.
7. Si la respuesta requiere acción (tool call), ejecutar la herramienta y realimentar el resultado al LLM.
8. Generar respuesta final al usuario.
9. Guardar la interacción en la memoria persistente.

- **Puntos de decisión:** ¿Cuándo preguntar al usuario? ¿Cuándo abortar? ¿Cuándo cambiar de LLM?
- **Handoff:** Si hay múltiples agentes, ¿cómo se delegan tareas entre orquestadores?
- **Recuperación de errores:** Retry budget (máximo 3 intentos), escalado a humano, logging en `.tmp/run_state.json`.

4.3. Esquema Backend

Define la **arquitectura interna** del agente.

- **System prompt:** Documento maestro con instrucciones, contexto del proyecto, registro de errores previos.
- **Tools/Skills:** Catálogo de funciones ejecutables (scraping, EDA, cálculos, generación de informes).
- **Memoria persistente:** Base de datos vectorial (ChromaDB) con embeddings de documentos y conversaciones.
- **Script orquestador (Layer 2):** Capa central que decide qué LLM invocar, construye los prompts con contexto (memoria + tools + directivas), ejecuta las llamadas a las APIs, valida y post-procesa las respuestas. Es el *middleware* entre el usuario y los LLMs.
- **Tools/Skills (Layer 3):** Funciones deterministas (scripts Python) que el orquestador invoca según la decisión del LLM: scraping, EDA, generación de informes, etc.
- **Estado de ejecución:** Archivo JSON de estado (`.tmp/run_state.json`) para trazabilidad y recuperación.

4.4. Plan de Implementación

Define el **cómo se construye** el agente de forma iterativa.

- **Fase 1 — Base:** System prompt + tool de diagnóstico + interacción CLI básica.
- **Fase 2 — Memoria:** Integración de RAG (ChromaDB + embeddings).
- **Fase 3 — Habilidades:** Incorporación de tools específicas (scraping, EDA, evaluación).
- **Fase 4 — Robustez:** Manejo de errores, logging, retry budget, auto-corrección.
- **Fase 5 — Refinamiento:** Iterar sobre la base de errores registrados, mejorar prompts, añadir nuevas capacidades.

5. Conclusión

Esta metodología separa la **intención** (PRD) de la **ejecución** (TRD, Backend) y la **experiencia** (UI/UX, App Flow), permitiendo iterar sobre cada capa sin acoplamiento excesivo. El resultado es un agente de IA cuyo comportamiento es predecible, depurable y mejorable de forma sistemática.